

Полиномиальная регрессия, тестирование и регуляризация с помощью Turi Create

В этом разделе рассмотрим пример полиномиальной регрессии с регуляризацией в Turi Create. Вот код для этого раздела.

- **Блокнот:** https://github.com/luisguiserrano/manning/blob/master/Chapter_4_Testing_Overfitting_Underfitting/Polynomial_regression_regularization.ipynb.

Мы начнем с набора данных, который показан на рис. 4.8. Видим, что кривая, которая лучше всего соответствует этим данным, представляет собой параболу, которая открывается вниз (грустный смайлик). Так что это не та задача, которую можно решить с помощью линейной регрессии, — мы должны использовать полиномиальную регрессию. Кроме того, применим регуляризацию для настройки нашей модели. Набор данных хранится в SFrame под именем `data`, его первые несколько строк приведены в табл. 4.1.

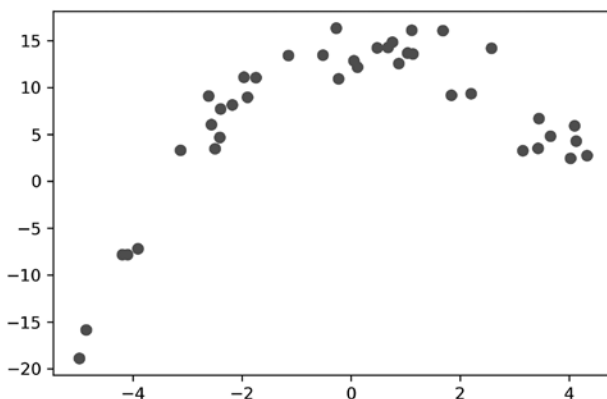


Рис. 4.8. Набор данных

Таблица 4.1. Первые четыре строки набора данных

<i>x</i>	<i>y</i>
3,4442185152504816	6,685961311021467
-2,4108324970703663	4,690236225597948
0,11274721368608542	12,205789026637378
-1,9668727392107255	11,133217991032268

Реализация полиномиальной регрессии в Turi Create состоит в том, чтобы добавить в исходный набор данных множество столбцов, соответствующих степеням основного признака, и применить линейную регрессию к расширенному

набору данных. Если основным признаком является, скажем, x , то добавляем столбцы со значениями x^2 , x^3 , x^4 и т. д. Таким образом, наша модель находит линейные комбинации степеней x , которые в точности являются полиномами в x . Если `SFrame`, содержащий данные, называется `data`, то для добавления столбцов для степеней до x^{199} мы применяем следующий код:

```
for i in range(2,200):
    string = 'x^'+str(i)
    data[string] = data['x'].apply(lambda x:x**i)
```

Первые несколько строк и столбцов результирующего набора данных приведены в табл. 4.2. Столбец x^k соответствует переменной x^k , где $k = 2, 3, 4$. Набор данных содержит 200 столбцов.

Таблица 4.2. Четыре верхние строки и пять крайних левых столбцов нашего набора данных

x	y	x^2	x^3	x^4
3,445	6,686	11,863	40,858	140,722
-2,411	4,690	5,812	-14,012	33,781
0,113	12,206	0,013	0,001	0,000
-1,967	11,133	3,869	-7,609	14,966

Теперь применим линейную регрессию к этому набору данных с 200 столбцами. Обратите внимание на то, что модель линейной регрессии здесь выглядит как линейная комбинация переменных в столбцах. Но поскольку каждый столбец соответствует одночлену, то полученная модель выглядит как многочлен по переменной x .

Прежде чем обучать какие-то модели, нужно разделить данные на обучающий и тестовый наборы данных, используя следующую строку кода:

```
train, test = data.random_split(.8)
```

Теперь набор данных разделен на обучающий набор `train` и тестовый набор `test`. В репозитории указано случайное начальное значение, поэтому мы всегда получаем одни и те же результаты, хотя на практике в этом нет необходимости.

Способ применения регуляризации в `Turi Create` прост: нам нужно лишь указать параметры `l1_penalty` и `l2_penalty` в методе `create` при обучении модели. Этот штраф является именно тем параметром регуляризации, который мы ввели в разделе «Регулирование уровня производительности и сложности модели...». Штраф 0 означает, что мы не используем регуляризацию. Таким образом, мы будем обучать три разные модели со следующими параметрами.

- Модель без регуляризации:
 - `l1_penalty=0`;
 - `l2_penalty=0`.

- Модель регуляризации L1:
 - `l1_penalty=0.1`;
 - `l2_penalty=0`.
- Модель регуляризации L2:
 - `l1_penalty=0`;
 - `l2_penalty=0.1`.

Обучим модели с помощью следующих трех строк кода:

```
model_no_reg = tc.linear_regression.create(train, target='y',
    l1_penalty=0.0, l2_penalty=0.0)
model_L1_reg = tc.linear_regression.create(train, target='y',
    l1_penalty=0.1, l2_penalty=0.0)
model_L2_reg = tc.linear_regression.create(train, target='y',
    l1_penalty=0.0, l2_penalty=0.1)
```

Первая модель не использует регуляризацию, вторая применяет регуляризацию L1 с параметром 0,1, а третья — регуляризацию L2 с параметром 0,1. Графики результирующих функций показаны на рис. 4.9. Обратите внимание на то, что на нем точки в обучающем наборе показаны кружками, а точки в тестовом наборе — треугольниками.

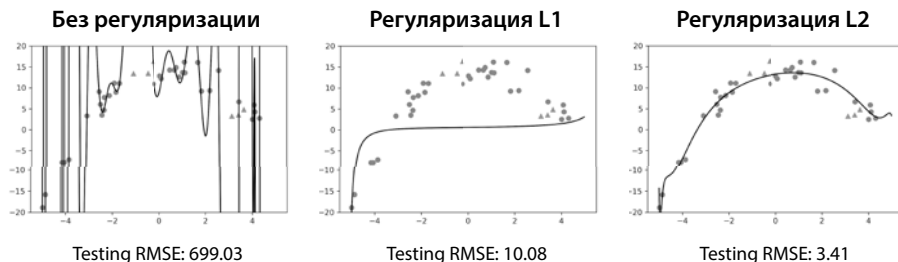


Рис. 4.9. Три модели полиномиальной регрессии для набора данных: слева — без регуляризации; по центру — регуляризация L1 с параметром 0,1; справа — регуляризация L2 с параметром 0,1

Обратите внимание на то, что модель без регуляризации довольно хорошо подходит для обучающих точек, но она хаотична и плохо работает для тестовых точек. Модель с регуляризацией L1 хорошо работает как с обучающими, так и с тестовыми наборами. Модель с регуляризацией L2 прекрасно справляется как с обучающими, так и с тестовыми наборами и, похоже, действительно отражает форму данных.

Также обратите внимание: во всех трех моделях граничная кривая несколько хаотична в конечных точках. Это вполне объяснимо, потому что конечные точки имеют меньше данных и для модели вполне естественно не знать, что делать, когда данных нет. Оценивать модели следует по тому, хорошо ли они работают внутри границ набора данных, и мы не должны ожидать, что модель будет хорошо работать за этими границами. Даже мы, люди, скорее всего, не

сможем строить хорошие прогнозы за пределами границ модели. Например, как, по-вашему, эта кривая будет выглядеть за пределами набора данных? Будет ли она продолжаться как парабола, которая открывается вниз? Будет ли колебаться вечно, как синусоидальная функция? Если мы этого не знаем, то не следует ожидать, что это знает модель. Таким образом, постарайтесь не обращать внимания на странное поведение кривой в конечных точках на рис. 4.9 и сосредоточьтесь на поведении модели внутри интервала, в котором расположены данные.

Чтобы найти ошибку тестирования, используем следующую строку кода с соответствующим именем модели. Эта строка возвращает максимальную ошибку и среднюю квадратическую погрешность:

```
model.predict(test)
```

Тестовые средние квадратические погрешности для моделей приведены далее:

- модель без регуляризации — 699,03;
- модель с регуляризацией L1 — 10,08;
- модель с регуляризацией L2 — 3,41.

Модель без регуляризации получила действительно большую среднюю квадратическую ошибку! Из двух других моделей намного лучше сработала модель с регуляризацией L2. Вот два вопроса, над которыми стоит задуматься.

1. Почему модель с регуляризацией L2 показала лучшие результаты, чем модель с регуляризацией L1?
2. Почему модель с регуляризацией L1 выглядит плоской, в то время как модели с регуляризацией L2 удалось отразить форму данных?

На эти два вопроса можно ответить одинаково, и, чтобы найти этот ответ, мы рассмотрим коэффициенты многочленов. Их можно получить с помощью следующей строки кода:

```
model.coefficients
```

Каждый многочлен имеет 200 коэффициентов, поэтому мы не будем отображать их все, но в табл. 4.3 вы видите первые пять коэффициентов для трех моделей. Что вы заметили? Надеюсь, то, что модель без регуляризации получила большие коэффициенты, модель с регуляризацией L1 — коэффициенты, очень близкие к 0, а модель с регуляризацией L2 — малые коэффициенты.

Таблица 4.3. Первые пять коэффициентов полиномов в трех моделях

Коэффициент	model_no_reg	model_L1_reg	model_L2_reg
$x^0 = 1$	8,41	0,5700	13,240
x^1	15,87	0,0700	0,870
x^2	108,87	-0,0040	-0,520
x^3	-212,89	0,0002	0,006
x^4	-97,13	-0,0002	-0,020

Взглянув на табл. 4.3, мы видим, что предсказания для трех моделей являются полиномами степени 200. Первые слагаемые выглядят следующим образом:

- модель без регуляризации — $\hat{y} = 8,41 + 15,87x + 108,87x^2 - 212,89x^3 - 97,13x^4 + \dots$
- модель с регуляризацией L1 — $\hat{y} = 0,57 + 0,07x - 0,004x^2 + 0,0002x^3 - 0,0002x^4 + \dots$
- модель с регуляризацией L2 — $\hat{y} = 13,24 + 0,87x - 0,52x^2 + 0,006x^3 - 0,02x^4 + \dots$

Из этих многочленов мы можем вывести следующее.

- Для модели без регуляризации все коэффициенты большие. Это означает, что многочлен хаотичен и не подходит для прогнозирования.
- Для модели с регуляризацией L1 все коэффициенты, за исключением постоянного (первого), малы — почти 0. Это означает, что при значениях, близких к нулю, многочлен очень похож на горизонтальную прямую $\hat{y} = 0,57$. Это лучше, чем предыдущая модель, но все же не очень подходит для составления прогнозов.
- Для модели с регуляризацией L2 коэффициенты уменьшаются с ростом степени, но все еще не так уж малы. Это дает вполне приличный многочлен для составления прогнозов.

Резюме

- Когда дело доходит до обучающих моделей, возникает множество проблем. Две проблемы, которые наблюдаются довольно часто, — это недообучение и переобучение.
- Недообучение возникает, когда мы используем для набора данных слишком простую модель. Переобучение происходит, когда применяемая модель слишком сложна.
- Эффективный способ выявить переобучение и недообучение — использование тестового набора данных.
- Чтобы протестировать модель, мы разделяем данные на два набора: обучающий и тестовый. Обучающий набор применяется для обучения модели, а тестовый — для ее оценки.
- Золотое правило машинного обучения заключается в том, чтобы никогда не задействовать тестовые данные для обучения или принятия решений в моделях.
- Контрольный набор — это еще одна часть набора данных, с помощью которой принимаем решения о гиперпараметрах модели.

- Недостаточно подходящая модель будет плохо работать на обучающем и контрольном наборах. Переобученная модель будет хорошо работать на обучающем наборе, но плохо на контрольном. Хорошая модель будет прекрасно работать как на обучающем, так и на контрольном наборе.
- График сложности модели используется для корректного определения ее сложности, чтобы она не переобучалась и не недообучалась.
- Регуляризация — крайне важный метод уменьшения переобучения в моделях машинного обучения. Он состоит в добавлении меры сложности (слагаемого регуляризации) к функции ошибки во время обучения.
- Нормы L1 и L2 — два наиболее распространенных показателя сложности, применяемых при регуляризации.
- Использование нормы L1 приводит к регуляризации L1, или регрессии лассо, нормы L2 — к регуляризации L2, или гребневой регрессии.
- Регуляризация L1 рекомендуется, когда набор данных содержит множество признаков и мы хотим свести многие из них к нулю. Регуляризацию L2 стоит применять, когда признаков в наборе данных мало и мы хотим сделать их маленькими, но не равными нулю.

Упражнения

Упражнение 4.1

Мы обучили четыре модели на одном и том же наборе данных с разными гиперпараметрами. В следующей таблице записали ошибки обучения и тестирования для каждой из моделей.

Модель	Ошибка обучения	Ошибка тестирования
1	0,1	1,8
2	0,4	1,2
3	0,6	0,8
4	1,9	2,3

- Какую модель вы бы выбрали для заданного набора данных?
- Какая модель выглядит так, будто она недостаточно хорошо соответствует данным?
- Какая модель выглядит так, будто она переобучена?

Упражнение 4.2

Имеется следующий набор данных.

x	y
1	2,0
2	2,5
3	6,0
4	14,5
5	34,0

Мы обучаем модель полиномиальной регрессии, которая предсказывает значение y как $\hat{y}$, где

$$\hat{y} = 2x^2 - 5x + 4.$$

Если параметр регуляризации $\lambda = 0,1$, а функция ошибки, которую мы использовали для обучения этого набора данных, представляет собой среднее абсолютное значение (MAE), определите следующее.

- А. Ошибку регрессии лассо модели (с помощью нормы L1).
- Б. Ошибку гребневой регрессии модели (с помощью нормы L2).